Лабораторная работа №07

Оптимизация вывода трехмерной сцены на экран.

Целью лабораторной работы №7 является реализация класса для работы со всей сценой (Scene). Данный класс позволит удобным способом работать со сценой, содержащей множество графических объектов, в частности, он позволит создавать один графический объект по его описанию во внешнем json-файле, а также загружать всю сцену из внешнего json-файла, в котором заданно количество и взаимное расположение всех графических объектов.

Кроме того, в данной седьмой лабораторной работе необходимо реализовать и продемонстрировать эффект, оказываемый тремя алгоритмами оптимизации:

1. Level of Detail - отсечение объектов в зависимости от их типа и дистанции до наблюдателя;
2. Frustum Culling - отсечение объектов не попадающий в области видимости камеры;
3. Geometry Instancing - сортировка графических объектов по материалам и одновременный вывод нескольких копий одинаковых моделей.

Описание моделей в json-файле.

В предыдущей лабораторной работе для демонстрации механизма наложения текстур или расчета освещения по Фонгу необходимо было создавать несколько графических объектов, для каждого из которых жестко назначался определенный материл, текстура и меш. При достаточно большом количестве графических объектов, мешей, текстур и материалов данный подход (определение взаимосвязей непосредственно в коде программы) становится весьма неудобным.

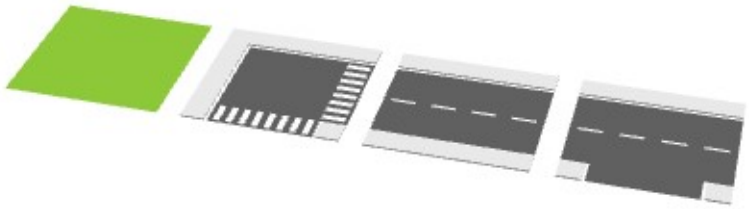
Другим вариантом организации программы является вынесение конфигурационных данных в отдельный файл. В частности, в рамках данной лабораторной работы определение того, как строится каждая модель, вынесено в отдельный json-файл – «models.json».

Файл «models.json» описывает все трехмерные модели. В вершине файла лежит json-объект (словарь/map), который состоит из множества пар ключ-значение. Ключ – это идентификатор модели, а значение – это полное описание данной модели. Для примера рассмотрим один из элементов json-объекта, который приведенный ниже:

```
"ambul": {
    "type": "vehicle",
    "dimensions": [ 3.574, 1.548, 1.54 ],
    "mesh": "data/meshes/vehicles/ambulance.obj",
    "texture": "data/textures/vehicles/ambulance.png",
    "material": "data/materials/dull_material.json"

}
```

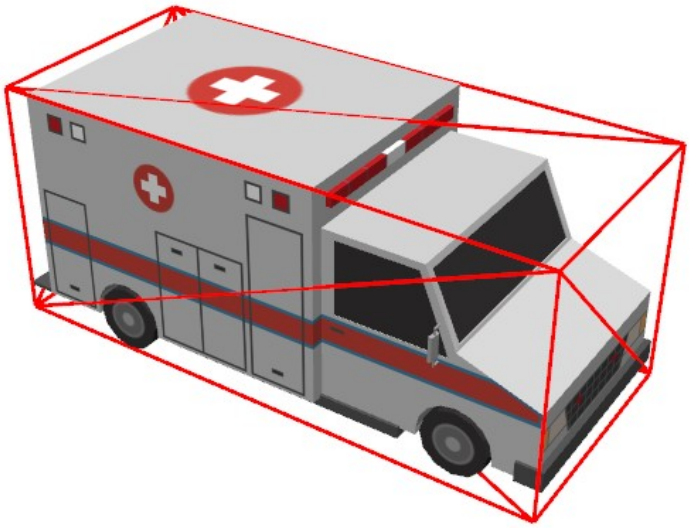
Данный элемент объекта описывает модель машины скоро помощи. Для обозначения модели используется идентификатор ambul (от английского ambulance). Таким образом, если программе потребуется создать графический объект на основе модели машины скорой помощи, ей необходимо будет найти элемент с ключом “ambul” и считать все его параметры.

Одним из параметров модели является её тип (type), который определяет к какой группе объектов относится данная модель. Тип модели играет важную роль для принятия решения о необходимости выводить объект на экран в алгоритме, основанном на использовании уровней детализации. В проекте различают следующие типы объектов:

Описание	**Примеры объектов**
Поверхность. Разные варианты дорог, асфальтового покрытия и травы. Всего 15 объектов. Для указания объектов этого типа используется строка “road”.	
Здания. Различные варианты зданий, в том числе, здания одной и той же формы, но с различной окраской. Всего 40 объектов. Для указания объектов этого типа используется строка “building”.	

Транспорт. Различные варианты легковых машин, автобусов, фургонов, грузовиков. Всего 21 объект. Для указания объектов этого типа используется строка “vehicle”.	
Большие объекты природы. Модели достаточно крупных деревьев. Всего 3 объекта. Для указания объектов этого типа используется строка “big nature”.	
Маленькие объекты природы. Небольшие объекты природы, к которым относятся камни, невысокие деревья и кусты. Всего 12 объектов. Для указания объектов этого типа используется строка “small nature”.	
Большие предметы окружения. Светофоры (двух видов), фонари, антенны и ветряные мельницы. Всего 5 объектов. Для указания объектов этого типа используется строка “big prop”.	
Средние предметы окружения. Сюда относится автобусная остановка, два билборда и вертолетная площадка. Всего 4 объекта. Для указания объектов этого типа используется строка “medium prop”.	
Маленькие предметы окружения. К маленьким предметам окружения относятся различные заборы, скамейки, урны и другие модели, размеры которых достаточно малы. Для указания объектов этого типа используется строка “small prop”	

Другим важным параметром, характеризующим модель, являются её габаритные размеры (dimensions), задающие размеры модели в виде вектора из трех компонент, которые соответствуют длине, высоте и глубине модели соответственно. Все модели, представленные в этом проекте, выравнены относительно центра локальной системы координат (системы координат модели), поэтому длина модели определяет её протяженность по оси X, высота – протяженности по оси Y и, наконец, глубина – протяженность по оси Z.



Габаритные размеры, представленные в json-файле, позволяют описать максимально плотный параллелепипед вокруг модели так, чтобы все вершины модели лежали строго внутри этого параллелепипеда. Подобный параллелепипед, полностью охватывающий модель и выравненный по осям координат этой модели, принято называть object oriented bounding box (OOBB, объектно-ориентированный ограничивающий параллелепипед).

Объектно-ориентированный ограничивающий параллелепипед (OOBB) является частным случаем ограничивающий объемов (bounding volumes). Ограничивающие объемы – это простейшие геометрические фигуры, полностью охватывающие трехмерный объект. Ограничивающие объемы используются для различных проверок вместо исходного, более сложного объекта. Другими вариантами ограничивающих объемов являются, например: сфера или параллелепипед, выравненный по осям глобальной системы координат (axis-aligned bounding boxes, AABB).

Наличие OOBB для каждого графического объекта позволяет оптимизировать некоторые алгоритмы вывода. В частности, в рамках данной лабораторной работе OOBB будет использоваться для реализации алгоритма frustum culling.

Другими параметрами модели являются:

- Поле mesh – определяет из какого файла должен быть загружен меш для построения модели.
- Поле texture – определяет из какого файла должна быть загружена текстура, которая будет накладываться на данную модель.
- Поле material – определяет какой файл должен быть использован для загрузки материала, применяемого к данной модели.

В связи со всем вышеперечисленным, в частности, в связи с необходимостью хранить дополнительные параметры графического объекта, необходимо внести изменения в класс GraphicObject. Прежде всего, необходимо определить перечисление (enum), для указания типа графического объекта в соответствии с полем type в json-представлении модели. Данное перечисление может выглядеть следующим образом:

```
// ТИП ДАННЫХ ГРАФИЧЕСКОГО ОБЪЕКТА
enum class GraphicObjectType { none, road, building, vehicle,
       big_nature, small_nature, big_prop, medium_prop, small_prop };
```

Сам класс для работы с графическим объектом будет выглядеть следующим образом:

```
// КЛАСС ДЛЯ РАБОТЫ С ГРАФИЧЕСКИМ ОБЪЕКТОМ
class GraphicObject
{
public:
	// конструктор по умолчанию
	GraphicObject();

	// установить различные параметры
	void setColor(glm::vec4 &color);
	void setPosition(glm::vec3 &position);
	void setAngle(float degree);
	void setType(GraphicObjectType type);
	void setDimensions(glm::vec3 &dimensions);

	// установить идентификаторы ресурсов
	void setMeshId(int id);
	void setTextureId(int id);
	void setMaterialId(int id);

	// получить различные параметры
	glm::vec4 getColor();
	glm::vec3 getPosition();
	float getAngle();
	GraphicObjectType getType();
	glm::vec3 getDimensions();

	// получить идентификаторы ресурсов
	int getMeshId();
	int getTextureId();
	int getMaterialId();

	// получить матрицу модели
	glm::mat4& getModelMatrix();

private:
	// цвет объекта
	glm::vec4 color;
	// тип модели
	GraphicObjectType type;
	// размер модели (W x H x D)
	glm::vec3 dimensions;
	// позиция
	glm::vec3 position;
	// угол поворота в горизонтальной плоскости в градусах, по часовой стрелке
	float angle;

	// матрица модели (задает позицию и ориентацию)
	glm::mat4 modelMatrix;

	// идентификатор используемого меша
	int meshId;
	// идентификатор используемой текстуры
	int textureId;
	// идентификатор используемого материала
	int materialId;

};
```

Описание сцены в json-файле.

Еще одной задачей лабораторной работы является вывод на экран не просто одного-двух графических объектов, а достаточно большого их количества, чтобы можно было протестировать различные алгоритмы оптимизации. В связи с этим, для определения состава сцены необходимо использовать данные, заданные в json-файле. Например, сцена, которая использовалась в предыдущей лабораторной работе может быть представлена следующим json-файлом:

```
[
    {
        "model": "big_tree",
        "position": [ 7.5, -0.75, 2.5 ],
        "angle": 0.0
    },
    {
        "model": "house_2_or",
        "position": [ 0.0, 0.0, 0.0 ],
        "angle": 0.0
    },
    {
        "model": "big_tree",
        "position": [ -7.5, -0.75, 2.5 ],
        "angle": 0.0
    },
    {
        "model": "police",
        "position": [ 4.5, -2.15, 6.5 ],
        "angle": -115.0
    },
    {
        "model": "jeep_grn",
        "position": [ -1.25, -2.15, 9.0 ],
        "angle": 170.0
    },
    {
        "model": "police",
        "position": [ 4.25, -2.15, 10.5 ],
        "angle": 105.0
    }
]
```

Как видно из вышеприведенного примера, сцена описывается массивом, каждый элемент которого представлен отдельным json-объектом, задающим графический объект. Каждый графический объект характеризуется:

- Именем модели (ключ «model»). Имя модели определяет её внешний вид в соответствии с описанием модели, заданном в файле «models.json», структура которого приведена в предыдущем разделе. Используя значения соответствующего ключа, необходимо считать параметры модели из json-представления и сформировать требуемый графический объект с необходимым мешем, материалом, текстурой и прочими параметрами.
- Позицией графического объекта (ключ «position»). Позиция графического объекта представлена списком из трёх вещественных значений - вектором (x, y, z), определяющим положение графического объекта в глобальной системе координат.
- Углом поворота (ключ «angle»). Угол поворота задает поворот в градусах по часовой стрелке в горизонтальной плоскости.

Подобный подход (разделение описания моделей и сцен) позволяет описывать сцену, состоящую из множества повторяющихся объектов, без необходимости повторно описывать все характеристики каждой модели. Кроме того, можно изменить внешний вид модели, заменив текстуру, окраску или даже геометрическую форму, не затрагивая сцену.

Реализация класса для работы со сценой (Scene).

Следующим этапом развития проекта является реализация класса для работы со сценой. Данный класс необходим для хранения всех объектов сцены, а также для выполнения определенных действий над сценой, например, для загрузки сцены из внешнего файла или для её вывода на экран. Класс сцены является одним из ключевых в данном проекте совместно с классами ResourceManager и RenderManager. В данной лабораторной работе класс должен иметь структуру, представленную ниже:

```
// КЛАСС ДЛЯ РАБОТЫ СО СЦЕНОЙ
class Scene
{
public:
	// конструктор по умолчанию
	Scene();

	// Инициализация сцены
	// параметр: json-файл с описанием моделей («models.json»)
	void init(std::string filename);

	// загрузка сцены из json-файла
	bool loadFromJson(std::string filename);

	// установка используемой камеры (ссылка на глобальный объект)
	void setCamera(Camera *camera);
	// установка используемого источника света (ссылка на глобальный объект)
	void setLight(Light *light);

	// Вывод всей сцены (посредством обращения к RenderManager)
	void draw();

	// Получение информации о сцене для отладки:
	// [количество объектов переданное на отрисовку]/[общее количество объектов сцены]
	std::string getSceneDescription();

private:
	// создание объекта по его идентификатору в json-файле ("models.json")
	GraphicObject createGraphicObject(std::string model);

	// определение, виден ли объект в зависимости от дистанции до него
	bool lodTest(GraphicObject& graphicObject);

	// определение, попадает ли объект в усечённую пирамиду видимости
	bool frustumCullingTest(GraphicObject& graphicObject);

private:
	// json-представление всех моделей (из файла "models.json")
	rapidjson::Document document;
	// указатель на камеру (на глобальную переменную)
	Camera *camera;
	// указатель на источник света (на глобальную переменную)
	Light *light;
	// все графические объекты
	std::vector<GraphicObject> graphicObjects;
	// статистика - сколько объектов передано на отрисовку в текущем кадре
	int renderedObjectCount;

};
```

Назначение полей и методов класса должно быть понятным по их названию, приведенным комментариям и функциям, которые возложены на класс Scene. Тем не менее, некоторые ключевые методы будут рассмотрены далее более подробно. Также следует напомнить, что при необходимости разрешается добавление дополнительных методов, как интерфейсных, так и вспомогательных.

Метод init.

Данный метод необходим, прежде всего, для загрузки json-файла описывающего все параметры всех моделей (в данном случае - это файл «models.json»). Метод вызывается один раз на этапе инициализации программы, загружает указанный файл, парсит его и помещает json-данные во внутренне поле класса для последующего использования при генерации новых графических моделей. Это позволит использовать уже готовые данные, а не загружать файл каждый раз при создании очередного графического объекта.

Для парсинга json-данных рекомендуется использовать библиотеку RapidJSON.

Метод createGraphicObject.

Данный метод используется для создания графического объекта на основании имени используемой модели. Например, если в качестве параметра указан идентификатор «ambul», это значит, что требуется создать графический объект, использующий модель машины скорой помощи. Параметры модели (используемый меш, материал, текстура, размеры модели и её тип) определяются во внешнем json-файле, что позволяет отделить программный код от данных и дает возможность изменять внешний вид моделей без повторной генерации кода. Структура json-файла была приведена в разделе «описание моделей в json-файле».

Следует отметить, что метод createGraphicObject не должен каждый раз заново загружать и парсить json-файл. Вместо этого необходимо использовать данные, ранее загруженные с помощью метода Init, что существенно увеличит скорость работы программы.

Метод loadFromJson.

Данный метод необходим для загрузки всей сцены из внешнего json-файла, структура которого была приведена в разделе «описание сцены в json-файле». Параметром функции является имя загружаемого файла сцены. В демонстрационном примере доступны три сцены:

- «demo_scene.json» - простейшая сцена, содержащая 6 объектов. Аналогичная сцена была использована в предыдущих лабораторных работах. При написании алгоритмов оптимизации рекомендуется начать именно с этой сцены, поскольку малое-количество объектов позволит легко убедиться, что, например, сортировка по материалам и текстурам выполняется корректно.
- «small_scene.json» - маленькая сцена, содержащая 95 объектов. Данная сцена может использоваться для проверки того, что программа корректно загружает более сложные сцены, а также в процессе написания алгоритмов оптимизации, поскольку сцена не слишком большая и даже без использования алгоритмов оптимизации даёт неплохое количество FPS.
- «big_scene.json» - большая сцена, содержащая 25680 объектов. Данную сцену необходимо использовать для демонстрации эффекта, оказываемого алгоритмами оптимизации, поскольку вывод этой сцены без алгоритмов оптимизации даёт чрезвычайно низкую производительность (FPS стремиться к нулю).

Стоит отметить, что при загрузке всей сцены необходимо создать все графические объекты, для чего используется вспомогательный метод createGraphicObject. Метод createGraphicObject возвращает созданный на основании имени модели графический объект, затем для графического объекта задается его позиция и угол поворота, после чего полученный графический объект добавляется в список графических объектов сцены (graphicObjects).

Метод draw.

Данный метод реализует «логический» вывод объектов сцены на экран.

«Логический» вывод объектов подразумевает, что класс Scene не будет вдаваться в подробности того, как графические объекты будут выведены на экран, какие низкоуровневые функции библиотеки OpenGL будут использоваться и какие шейдеры необходимы.

Например, метод draw может проверять, надо ли выводить объекты на экран или нет, и, в зависимости от этого, может передавать или не передавать их в класс RenderManager. В частности, объекты могут не передаваться в рендер-менеджер, если они не попадают в область видимости наблюдателя, находятся слишком далеко от него или еще по каким-то критериям не интересны пользователю.

При таком способе организации программы происходит отделение логики принятия решения о том, что объект необходимо вывести на экран, от низкоуровневых алгоритмов вывода. Для лучшего понимания логики взаимодействия классов Scene и RenderManager ниже приводится полный текст метода draw:

```
// Вывод всей сцены (посредством обращения к RenderManager)
void Scene::draw()
{
	// начинаем вывод сцены в новом кадре
	renderedObjectCount = 0;

	// добавляем в очередь все объекты, которые необходимо вывести с учетом оптимизации
	for (auto& grObj : graphicObjects) {
		// если объект не прошел тест на уровень детализации, то его не выводим
		if (!lodTest(grObj)) continue;
		// если объект не прошел frustum culling, то его не выводим
		if (!frustumCullingTest(grObj)) continue;
		// иначе добавляем объект в очередь на отрисовку
		renderedObjectCount++;
		RenderManager::instance().addToRenderQueue(grObj);
	}
}
```

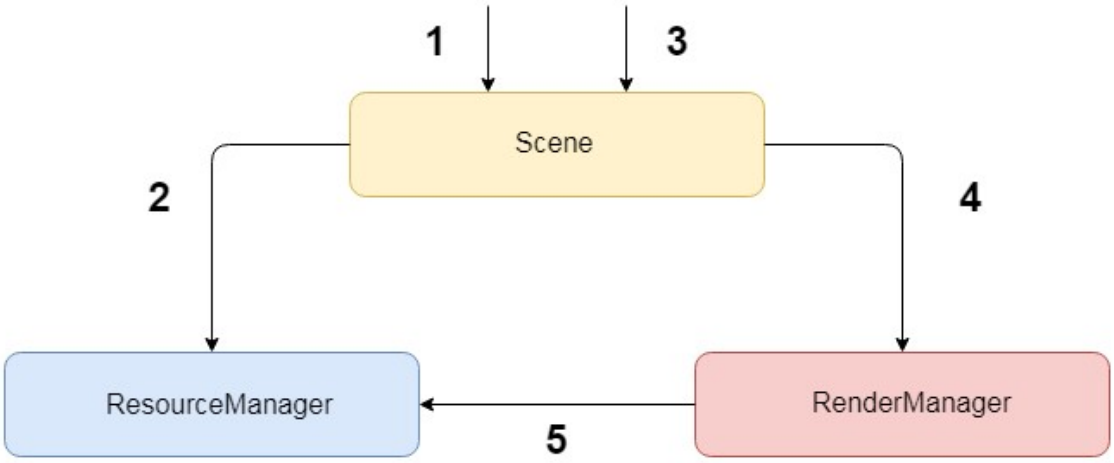
В свою очередь, класс RenderManager, получив список объектов, которые необходимо вывести на экран применяет все известные ему алгоритмы для их оптимального вывода. Данный подход позволяет более тонко настраивать систему, например, на ходу меняя способ фильтрации текстур или алгоритмы сглаживания ребер полигонов (anti-aliasing).

Еще одной причиной отделение класса Scene от класса RenderManager, отвечающего за вывод изображения на экран, является то, что не только трехмерные объекты нуждаются в отрисовке. На экран также может быть необходимо вывести элементы интерфейса, которые не относятся к сцене напрямую и хранятся отдельно (в данном проекте элементы интерфейса не реализованы). Для лучшего понимания ниже приводится полный текст функции display, отвечающей за построение одного кадра:

```
// функция вызывается при перерисовке окна
// в том числе принудительно, по команде glutPostRedisplay
void display()
{
	// для удобства определяем ссылку на RenderManager
	RenderManager& renderManager = RenderManager::instance();

	// начинаем вывод нового кадра
	renderManager.start();

	// выводим сцену
	scene.draw();

	// завершаем построение кадра
	renderManager.finish();

	// меняем передний и задний буферы цвета
	glutSwapBuffers();

	// вычисление FPS и его вывод в заголовок окна
	char windowTitle[128];
	int FPS = getFps();
	sprintf_s(windowTitle, 128, "Laba_07 [%i FPS][%s]", FPS,
		scene.getSceneDescription().c_str());
	glutSetWindowTitle(windowTitle);
}
```

Взаимодействие основных классов

Для лучшего понимания структуры проекта необходимо еще раз рассмотреть взаимосвязь между классами RenderManager, Scene и ResourceManager. Эти три класса являются теми тремя китами, на которых строится данный проект:



Взаимодействие классов осуществляется по следующей схеме:

1. **Загрузка сцены.** При инициализации программы происходит загрузка всей сцены из внешнего json-файла. Загрузка сцены подразумевает создание новых графических объектов, что требует обращения к менеджеру ресурсов.
2. **Загрузка нового ресурса.** При создании нового графического объекта класс Scene обращается к менеджеру ресурсов и получает от него идентификаторы загруженных ресурсов, в частности: идентификатор используемого меша, текстуры и материала.
3. **Вывод сцены.** В определенные моменты времени, например, после каждого этапа симуляции или при изменении размеров окна, требуется вывести сцену на экран. Для этого в функции display вызывается метод draw сцены, который, для выполнения поставленной задачи, обращается к классу RenderManager.
4. **Передача объектов для вывода.** Метод draw сцены не обращается к функциям OpenGL непосредственно и не занимается какой-либо низкоуровневой оптимизацией, а просто передает в класс RenderManager список объектов, который необходимо вывести. После этого за вывод объектов на экран отвечает уже класс RenderManager. В частности, данный класс должен выбрать шейдер, привязать текстуру, передать в шейдер различные параметры и вывести меш, передав все атрибуты каждой из его вершин в OpenGL.
5. **Обращение к ресурсу для его вывода на экран.** Для вывода объектов на экран классу RenderManager требуется получить определенные ресурсы. Например, каждый объект имеет идентификатор, показывающий, какой меш используется для представления этого объекта. Используя данный идентификатор, происходит обращение к классу ResourceManager, считывается меш и, вызывая соответствующий метод меша, объект выводится на экран.

Методы оптимизации приложений интерактивной трехмерной компьютерной графики.

Основной задачей приложений интерактивной трехмерной графики является создание максимально качественного изображения при сохранении достаточной производительности, обеспечивающей генерирование минимально допустимого количества кадров в секунду. Минимальным считается значение в 20-30 кадров в секунду (FPS, frame per second). Наличие чрезмерно низкого количества FPS превращает приложение в слайд-шоу, что делает некомфортным (или даже невозможным) использование такого приложения.

Одним из критериев качества изображения является сложность используемых моделей. Здесь, прежде всего, речь идет о сложности геометрической формы – меша. Чем большее количество полигонов используется в меше, тем более сглаженной и высоко детализированной получается модель. В связи с этим, различают высокополигональные модели (hi-poly) и низкополигональные модели (low-poly).

Наличие большого количества полигонов позволяет использовать в модели большее количество деталей, в результате чего модель более точно соответствует реальному или вымышленному прототипу. Тем не менее, не следует забывать, что каждая вершина модели при выводе на экран будет обрабатываться вершинным шейдером, что, естественно, требует дополнительных ресурсов видеокарты. В результате, при проектировании моделей необходимо найти баланс между качеством моделей и скоростью их обработки.

На производительность приложения также оказывает влияние и качество текстур. Текстуры высокого разрешения (HD текстуры) позволяют изобразить на объекте большее количество деталей, делая объекты менее размытыми, однако налагают дополнительные требования к количеству памяти видеокарты и скорости обработки.

На рисунке снизу показан пример переработки моделей в популярной игре Warcraft III:

Таким образом, одним из вариантов повышение производительности приложений является уменьшение качества моделей, что не является оптимальным решением. В настоящее время существует ряд алгоритмов, которые способны увеличить производительность, не затрагивая качество моделей. Все эти алгоритмы можно разделить на три большие группы, в зависимости от основной идеи, заложенной в основе предложенных оптимизаций:

1. **«Все что делать не обязательно, делать не следует».** В основе этой группы алгоритмов лежит простая идей, которая говорит о том, что если какие-то действия не оказывают результата на итоговое изображение, то не следует тратить процессорное время или время видеокарты на их выполнение.

   Классическим примером алгоритма этой группы является Frustum Culling – алгоритм, который определяет, попадает ли объект в область видимости камеры. В случае, если объект не попадает в область видимости камеры, он отбрасывается, то есть не передается на отрисовку. В результате видеокарта не тратит свое время на обработку всех вершин данного объекта только для того, чтобы понять, что ни одна вершина не отображается на экране.

2. **«Все что должно быть сделано, должно быть сделано максимально быстро».** В основе алгоритмов этой группы лежит простая идей, что все действия, которые действительно оказывают влияние на итоговое изображение, должны быть выполнены максимально быстро. Для этого программа должна быть хорошо оптимизирована, использовать наиболее подходящие структуры данных, а также учитывать архитектуру центрального процессора и возможности, которыми обладают современные видеокарты.

   Одним из примеров алгоритмов данной группы является использование буферов VBO. Использование данного механизма не влияет на результирующее изображение, но позволяет выводить объекты гораздо более эффективно.

   Другим примером является использование массива индексов, что позволяет получить преимущества от Post-TnL кеша видеокарты.

   И наконец, можно также упомянуть оптимизацию вывода геометрии с использованием инстансинга геометрии и минимизация количества батчей.

3. **«Если не видно разницы, зачем платить больше?».** Последняя группа алгоритмов является специфичной для трехмерной компьютерной графики и эксплуатирует несовершенство человеческого зрения. В частности, нет смысла использовать более затратные методы построения изображения или более сложные модели, если разница будет не столь заметна для наблюдателя.

   К данной группе алгоритмов относятся, например, такие техники создания модели как bump mapping, normal mapping или parallax mapping, а также такой прием, как использование уровней детализации (Levels Of Detail, Lod).

Для лучшего понимания некоторых техник, а также сложностей, с которыми сталкиваются разработчики, рекомендуется посмотреть видео, подготовленное работниками компании Wargaming.net, в котором они описывают настройки графики и приемы оптимизации в игре World of Tanks:

https://www.youtube.com/watch?v=N22cMOqjx0k

Использование уровней детализации (LoD) для оптимизации вывода объектов сцены.

В данной лабораторной работе, чтобы продемонстрировать эффект алгоритмов оптимизации, необходимо работать с достаточно большой сценой. Так сцена, описанная в файле «big_scene.json», содержит 25680 объектов. В результате, при запуске программы без реализации каких-либо алгоритмов оптимизации, она показывает весьма скромные результаты, количество FPS в этом случае равно 14, а на экран выводятся все 25680 объектов сцены:

При этом большая часть объектов настолько удалена от наблюдателя, что рассмотреть их детали становится практически невозможным. Тем не менее, каждый из этих 25680 объектов по-прежнему передается в OpenGL, каждая вершина каждого объекта обрабатывается вершинным шейдером, после чего вычисляются цвета всех фрагментов. Зачастую для сильно удаленных объектов количество сгенерированных фрагментов будет очень мало, 5-10 пикселей на каждый объект. Возникает резонный вопрос, стоит ли выводить этот объект, если он занимает площадь в 10 пикселей?

Особенностью программ в области компьютерной графики является то, что конечный результат предназначен для восприятия человеком. В связи с этим, если вывод некоторых объектов почти не сказывается на итоговой картинке (не различим для человеческого глаза, или является не столь существенным), можно вовсе отказаться от их вывода на экран.

На рисунке снизу приведена та же сцена, что и ранее, но малозначимые объекты (прежде всего заборы, урны, кусты и камни), расположенные на достаточно большом удалении от наблюдателя не выводятся на экран. Это позволило уменьшить количество объектов, передаваемых для отрисовки, до 17673 штук практически без изменения итогового изображения. При этом количество FPS увеличилось с 14 до 20, что в абсолютных значения не очень много, но в относительных единицах дало прирост производительности в 42.8 процента:

Данный подход (отказ от вывода объекта или вывод упрощенной версии объекта) называется использованием уровней детализации объекта (Level of Detail, LoD). Использование данного подхода подразумевает, что при выводе сцены, для каждого объекта рассчитывается расстояние от наблюдателя до объекта. В случае если объект является малозначимым, а расстояние до него достаточно большим, объект можно не выводить. Хороший пример, демонстрирующий комплексное применение данной техники в игре Watch Dog, приведен по следующей ссылке:

http://international.download.nvidia.com/geforce-com/international/comparisons/watch-dogs/watch-dogs-level-of-detail-comparison-1-lod-ultra-vs-lod-low.html

В данном проекте значимость объекта определяется его типом, который считывается из файла «models.json» и кодируется соответствующей строкой. Типы моделей были рассмотрены в разделе «описание моделей в json-файле» и еще раз перечислены в таблице:

Тип объекта	**Кодировка в xml-файле.**	**Значение в перечислении GraphicObjectType**	**Расстояние, при котором выводится объект.**
Поверхность - различные варианты дорог, травы и асфальтового покрытия.	type: "road"	road	Объекты данного типа должны выводиться при любой дистанции до наблюдателя.
Здания – различные виды зданий, в том числе, одной и той же формы, но с различной окраской.	type: "building"	building	Объекты данного типа должны выводиться при любой дистанции до наблюдателя.
Транспорт - различные варианты машин, грузовиков и автобусов.	type: "vehicle"	vehicle	Объекты выводятся, если дистанция не превышает 500 единиц.
Большие объекты природы – несколько моделей крупных деревьев.	type: "big nature"	big_nature	Объекты выводятся, если дистанция не превышает 350 единиц.
Маленькие объекты природы – менее значимые объекты природы: кусты и камни.	type: "small nature"	small_nature	Объекты выводятся, если дистанция не превышает 200 единиц.
Большие предметы окружения – светофоры, фонари и некоторые другие заметные предметы окружения.	type: "big prop"	big_prop	Объекты выводятся, если дистанция не превышает 400 единиц.
Средние предметы окружения, например, билборды или автобусные остановки.	type: "medium prop"	medium_prop	Объекты выводятся, если дистанция не превышает 300 единиц.
Мелкие предметы окружения –объекты, которые почти незаметны на большом удалении от наблюдателя.	type: "small prop"	small_prop	Объекты выводятся, если дистанция не превышает 200 единиц.

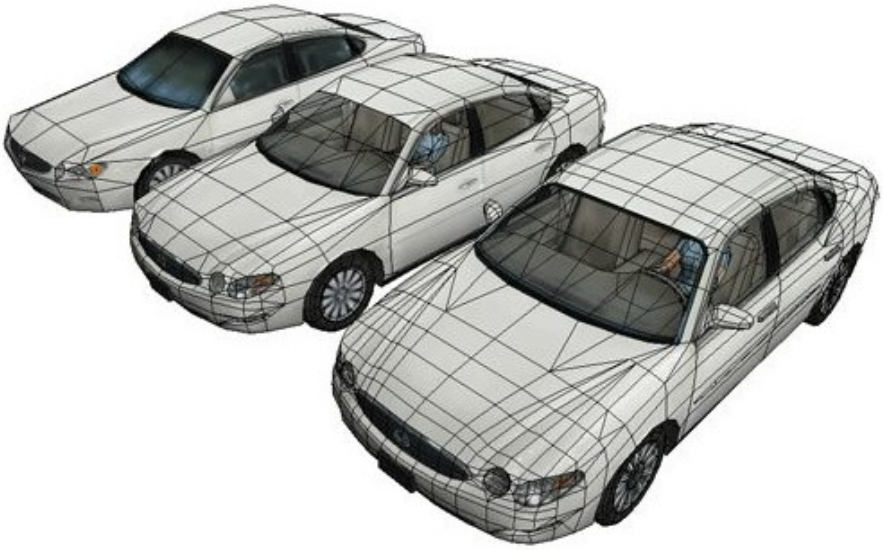
В данной лабораторной работе необходимо организовать вывод объектов на экран с учетом их типа и расстояния от наблюдателя до объекта. Для этого в классе для работы со сценой (Scene) для каждого объекта осуществляется проверка на необходимость его отрисовки. В класс RenderManager передаются только те объекты, которые прошли проверку и должны быть выведены на экран.

Необходимо отметить, что вычисление дистанции от наблюдателя до объекта, а также анализ типа объекта – это дополнительная нагрузка на процессор. В некоторых случаях эта дополнительная работа может нивелировать выгоду, полученную в результате того, что часть объектов не выводится на экран. Это зависит от того, что выполняется быстрее – дополнительная проверка средствами центрального процессора, или преобразования объекта средствами видеокарты, даже если в итоге он не будет виден на экран. Иными словами, необходимо определить, что является узким горлышком системы (bottleneck) - процессор или видеокарта. Иногда бывает выгоднее вывести модель на экран в любом случае, нежели выполнять дополнительные проверки.

Тем не менее, в случае если модель является достаточно сложной, например, состоит из большого количества полигонов или использует сложный шейдер расчета освещения, использование уровней детализации практически всегда является оправданным.

Следует также отметить, что расстояние от наблюдателя до объекта может использоваться и для других целей. Например, если расстояние слишком большое (больше 1200 единиц), то объект гарантированно не попадет в усеченную пирамиду видимости и его можно отбросить сразу, без учета его типа.

В некоторых случаях часть объектов является настолько важной с точки зрения логики работы программы, что эти объекты нельзя отбрасывать, даже если они находятся очень далеко от наблюдателя. Вместе с тем и вывод объекта, состоящего из нескольких тысяч полигонов, на большом расстоянии является неоправданным. В этом случае, для таких объектов создается несколько моделей, каждая из которых отличается детализацией, и, в зависимости от дистанции, выбирается модель необходимого качества:



Данный подход, однако, требует не только большего объема памяти для хранения всех уровней детализации для каждого объекта, но и тщательного подбора параметров, определяющих момент переключения с одного уровня детализации на другой, чтобы это было как можно менее заметно. Кроме того, данные модели необходимо сгенерировать, что требует больших временных затрат на этапе разработки программы.

Использование уровней детализации обладает еще одним важным преимуществом – гибкостью настройки. Каждый пользователь может индивидуально включить, отключить или настроить параметры данного алгоритма, в зависимости от аппаратных возможностей своего компьютера. Например, на более современных компьютерах можно увеличить дистанцию, при которой происходит переключение уровней детализации, или вовсе отказаться от этого режима, получив максимально качественное изображение. В тоже самое время, возможность уменьшения количества объектов, пусть даже в ущерб качеству изображения, позволит расширить спектр устройств, на которых программа может быть запущена.

Использование техники LoD порождает, в том числе, и некоторые забавные ситуации. Например, существует расхожее мнение в превосходстве персональных компьютеров над игровыми приставками, которое может быть проиллюстрировано следующей шуткой:

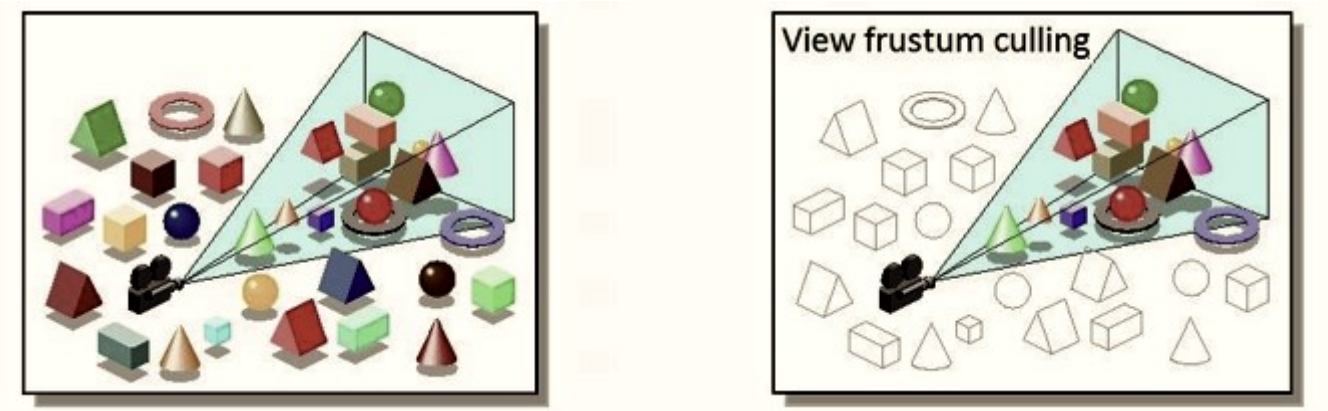
- Why did the console peasant cross the road?
- To render the building on the other side.

Реализация алгоритма frustum culling.

В результате использования уровней детализации количество объектов, выводимых в каждом кадре, удалось уменьшить с 25 тысяч до 17 тысяч, одновременно увеличив значение FPS. При этом совершенно очевидно, что не все эти объекты действительно попадают в поле зрение наблюдателя (в усеченную пирамиду видимости) и видны на экране.

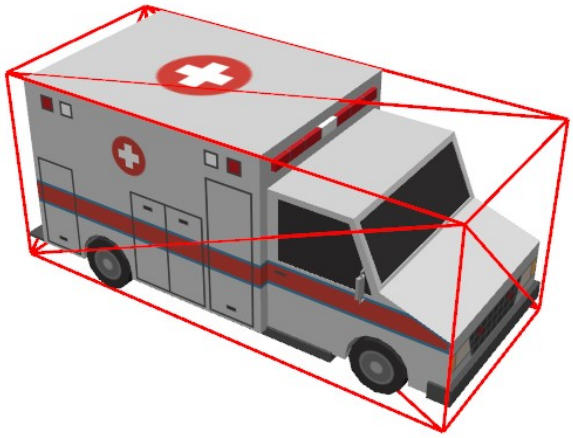
Библиотек OpenGL реализует процесс рендеринга таким образом, чтобы те вершины, которые лежат вне усеченной пирамиды видимости (frustum), были отброшены и не выводились на экран. Тем не менее, для того чтобы определить, лежит ли вершина в области видимости усеченной пирамиды, необходимо передать все вершины в OpenGL. Вершинный шейдер обработает каждую вершину и только после этого будет понятно, видна ли вершина на экране. Для оптимизации вывода мешей было бы неплохо заранее определить, попадает ли объект в усеченную пирамиду видимости и в случае, если он не попадает в эту пирамиду, даже не пытаться выводить его на экран. Данный алгоритм получил название «усечение по пирамиде видимости» или Frustum Culling.

Алгоритм frustum culling предусматривает проверку попадания объекта в усеченную пирамиду видимости и отбрасывание тех объектов, которые не попадают в эту пирамиду, то есть такие объекты даже не передаются в OpenGL и их вершины не преобразовываются вершинным шейдером:



Однако здесь возникает определенная проблема: для точного определения, попадает ли объект в область видимости камеры, необходимо проверить все его вершины, то есть преобразовать вершины по тем же самым алгоритмам, что используются в вершинном шейдере. Таким образом, каждая вершина по-прежнему должна быть преобразована с использованием матрицы модели, матрицы камеры и матрицы проекции. Более того, данные действия будут выполняться не на видеокарте, а на центральном процессоре, который справляется с этой задачей гораздо хуже.

Для того чтобы оптимизировать дынный процесс вместо тестирования каждой вершины объекта происходит тестирование более простых форм, которые называются ограничивающими объемами (bounding volumes).



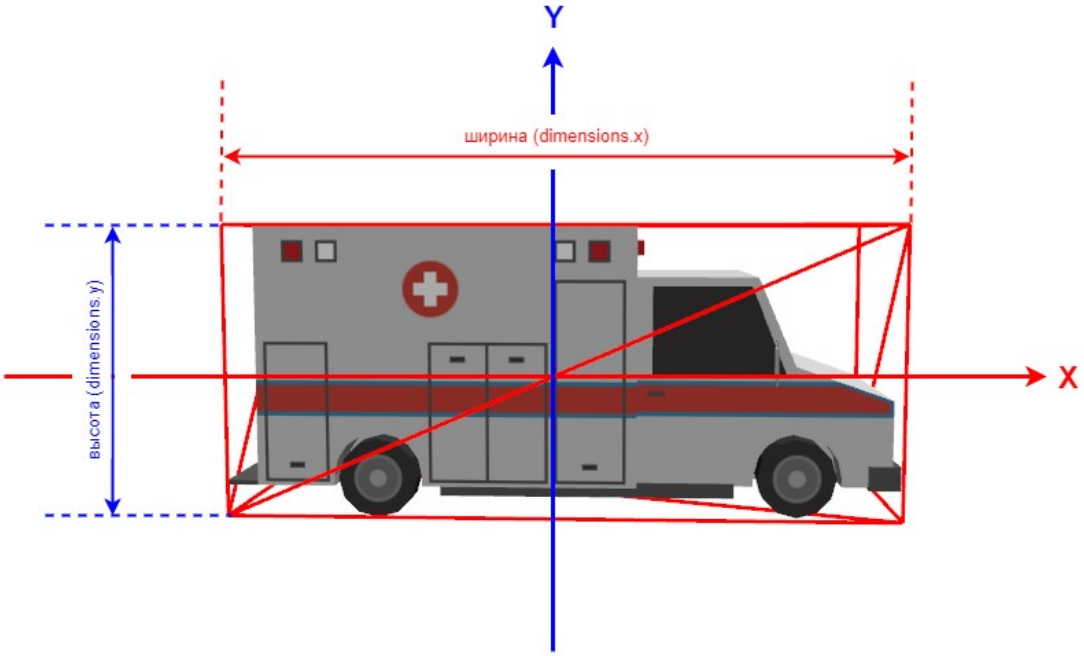
Ограничивающие объемы – это простые геометрические формы (например, сфера или параллелепипед) которые полностью включают в себя все вершины исходного объекта. Справа представлен пример такого объема, заданного с помощью параллелепипеда, ориентированного по осям системы координат объекта (object oriented bounding box - OOBB или просто oriented bounding box – OBB).

В результате, вместо тестирования нескольких тысяч или даже десятков тысяч вершин исходного объекта, тестированию подвергаются более простые геометрические формы. Например, для проверки попадания параллелепипеда в усеченную пирамиду видимости необходимо протестировать всего восемь вершин. В результате выполнения тестирования возможна одна из следующих ситуаций:

- Если оказывается, что ограничивающий объем лежит вне усеченной пирамиды видимости, то очевидно, что и сам объект (который полностью находится внутри ограничивающего объема) не будет виден наблюдателю;
- Если ограничивающий объем полностью лежит внутри усеченной пирамиды видимости (все вершины ограничивающего объема лежат внутри), то и сам объект виден наблюдателю;
- Ну и наконец, если ограничивающий объем виден лишь частично, то считается, что объект виден наблюдателю. Это допущение является не совсем корректным, поскольку, существует вероятность того, что наблюдатель будет видеть только маленький угол ограничивающего объема, в котором нет никаких частей реального объекта. Тем не менее, в этом случае проще вывести объект на экран, чем реализовывать другие, более сложные и точные проверки.

В рамках данной лабораторной работы в качестве ограничивающих объемов будет использоваться object oriented bounding box, параметры которого заданы в файле «models.json» в виде габаритных размеров в длину (первая компонента: ось X системы координат модели), ширину (вторая компонента: ось Y системы координат модели) и глубину (третья компонента: ось Z системы координат модели).

Пример, иллюстрирующий связь габаритных размеров (dimensions), полученных из файла, и OBB, используемого для тестирования, приведен на следующем рисунке (ось Z не показана):



Также следует отметить, что все модели, используемые в данном проекте, являются выровненными относительно центра локальной системы координат. В результате, ограничивающий объем является параллелепипедом, вершины которого заданы следующим образом:

```
(+ dimensions.x / 2, + dimensions.y / 2, + dimensions.z / 2, 1.0);
(+ dimensions.x / 2, + dimensions.y / 2, - dimensions.z / 2, 1.0);
(+ dimensions.x / 2, - dimensions.y / 2, + dimensions.z / 2, 1.0);
(+ dimensions.x / 2, - dimensions.y / 2, - dimensions.z / 2, 1.0);
(- dimensions.x / 2, + dimensions.y / 2, + dimensions.z / 2, 1.0);
(- dimensions.x / 2, + dimensions.y / 2, - dimensions.z / 2, 1.0);
(- dimensions.x / 2, - dimensions.y / 2, + dimensions.z / 2, 1.0);
(- dimensions.x / 2, - dimensions.y / 2, - dimensions.z / 2, 1.0);
```

Необходимо отметить, что вершины ограничивающего объема заданы в системе координат модели, поэтому, для преобразования в глобальную систему координат их необходимо умножить на ту же самую матрицу модели, которая используется для преобразования самой модели.

Проверку того, попадает ли вершина OBB в область усеченной пирамиды видимости можно легко выполнить если вспомнить, как эту задачу решает OpenGL. Для этого вершину необходимо умножить сначала на матрицу модели, затем на матрицу наблюдателя и, наконец, на матрицу проекции. Полученное значение является значением в усеченной системе координат (clip space). Вершина видна наблюдателю, если каждая компонент полученного вектора (x, y и z) лежит в интервале [-w; +w].

Схематично, алгоритм проверки видимости OBB для каждого графического объекта в составе сцены может быть представлен следующей последовательностью действий:

1. Сформировать матрицу перевода вершин OBB в систему нормализованных координат устройства. Для этого необходимо, используя матрицу проекции, матрицу наблюдения и матрицу модели, вычислить матричное произведение в следующей последовательности:

```
// матрица преобразований в усеченную систему координат (clip space)
mat4 PVM = camera->getProjectionMatrix() * camera->getViewMatrix()
             * graphicObject.getModelMatrix();
```

2. Используя полученную матрицу перевести все 8 вершин, задающих углы OBB, в усеченную систему координат. Для этого необходимо вычислить произведение полученной матрицы на вектор координат вершины (вектор умножается на матрицу справа, то есть: PVM * vector).

3. Используя полученные восемь вершин, необходимо определить, виден ли OBB наблюдателю. В частности:

   a. Если все вершины лежат справа от усеченной пирамиды видимости (x > +w), то OBB не виден;

   b. Если все вершины лежат слева от усеченной пирамиды видимости (x < -w), то OBB не виден;

   c. Если все вершины лежат выше усеченной пирамиды видимости (y > +w), то OBB не виден;

   d. Если все вершины лежат ниже усеченной пирамиды видимости (y < -w), то OBB не виден;

   e. Если все вершины лежат дальше, чем дальняя плоскость отсечения (z > +w), то OBB не виден;

   f. Если все вершины лежат ближе, чем ближняя плоскость отсечения (z < -w), то OBB не виден;

   g. В противном случае OBB виден и, соответственно, весь объект должен быть отправлен на отрисовку. Этот пункт включает в себя, в том числе, ситуацию, при которой, например, часть вершин лежит слева от усеченной пирамиды видимости, а часть – справа. В этом случае, несмотря на то что ни одна вершина ограничивающего объема непосредственно не попадает в область видимости, объект пересекает усеченную пирамиду видимости и, таким образом, виден наблюдателю.

Следует отметить, что алгоритм frustum culling хорошо работает совместно с использованием уровней детализации, однако следует учесть один небольшой нюанс. Поскольку вычисление расстояние от наблюдателя до объекта является более простой операцией, чем матричные преобразования, при совместном использовании этих техник оптимизации рекомендуется придерживаться следующей последовательности действий:

1. Вначале выполняется проверка объекта с учетом уровней детализации. Если объект является малозначимым и не должен выводится на экране, то он отбрасывается и не подвергается дальнейшей проверке, что позволит сэкономить процессорное время.

2. Если объект прошёл предыдущую проверку, то он дополнительно подвергается проверке на попадание в область видимости. Если он не проходит эту проверку, то он отбрасывается.

3. Если объект прошёл все проверки, то он передается в RenderManager для отрисовки, где будут применены дополнительные методы оптимизации, например, сортировка по материалам и instancing.

В результате применения алгоритма Frustum Culling совместно с использованием уровней детализации, как демонстрирует приведенный ниже скриншот, удалось существенно сократить количество объектов, переданных на отрисовку. В частности, на отрисовку передается только 1405 объектов из 25680, что привело к увеличению FPS с 20 до 54 (прирост производительности равен 170%). Прирост FPS мог бы быть больше, если бы использовались более сложные объекты.

Минимизация количества обращений к видеокарте и использование geometry instancing'a.

Уменьшение количества обращений к видеокарте.

Помимо количества полигонов и размера текстур на скорость построения изображения оказывают влияние еще множество факторов, одним из которых является количество обращений к видеокарте. Обращение к видеокарте подразумевает взаимодействие с ядром операционной системы и с драйвером видеокарты, что требует дополнительных затрат. Поэтому, чем меньше обращений к видеокарте происходит за время построения одного кадра, тем быстрее осуществляется данный процесс.

Вопрос уменьшения количества обращений в видеокарте уже был затронут при написании класса для работы с шейдером (Shader). В частности, в этом классе при установке значения uniform-переменной необходимо знать её индекс, который можно получить с помощью функции glGetUniformLocation. Тем не менее, запрос этого индекса каждый раз является нерациональным, поскольку он не меняется после линковки шейдерной программы. Поэтому индекс переменной запрашивается только один раз, при первом обращении к указанной переменной. При всех последующих обращениях индекс переменной берется из внутреннего ассоциативного контейнера, что не требует участия операционной системы.

Обращение к видеокарте происходит не только в момент запроса индекса uniform-переменной, но и во многих других случаях. Для демонстрации этого утверждения, рассмотрим существующую схему вывода объектов, реализованную в классе RenderManager. В частности, построение изображения одного кадра выполняется в соответствии со следующей последовательностью действий:

1. Выбор необходимого шейдера.
2. Установка общих uniform-переменных. К таким переменным относится матрица проекции и параметры источника света. При установке каждой переменной её значение передается в видеокарту, то есть происходит обращение к видеокарте, как, в принципе, при установке любой uniform-переменной.
3. Вывод каждого графического объекта происходит по одной и той же схеме и разбивается на следующие этапы:
    a. Установка параметров материала для этого объекта. Установка параметров материала подразумевает установку uniform-переменных, отвечающих за параметры материала в соответствии с формулой Фонга.
    b. Привязка необходимой текстуры к текстурному блоку, что подразумевает, также, установку значение uniform-переменной типа sampler, для доступа к текстурному блоку из шейдера.
    c. Установка матрицы наблюдения модели для этого объекта. Для каждого объекта вычисляется своя собственная матрица наблюдения модели, которая затем передается в видеокарту в виде uniform-переменной.
    d. Вывод меша для данного объекта. В завершении, необходимо вывести сам меш, привязав необходимый VAO-объект и вызвав одну из draw-функций (glDrawArrays или glDrawElements).

В процессе отрисовки графических объектов происходит весьма интенсивное взаимодействие с видеокартой во время установки всех uniform-переменных. Данная последовательность действий весьма хороша с обучающей точки зрений, поскольку она проста, понятна и гарантирует, что программист не забудет установить значение uniform-переменных или привязать текстуру к текстурному блоку.

Следует отметить, что объекты передаются в произвольном порядке, поэтому заранее неизвестно сколько будет объектов, в каком порядке они будут переданы или какие материалы они будут использовать. Приведем пример отрисовки пяти объектов, некоторые из которых имеют одинаковый материал (непосредственно материал + используемая текстура), в последовательности, которую схематично можно представить следующим рисунком:

Совершенно очевидно, что приведенный выше алгоритм вывода объектов будет довольно неэффективным по нескольким причинам. Во-первых, несмотря на то, что первые два объекта используют один и тот же материал, установка соответствующих uniform-переменных будет осуществлена два раза. С учетом того, что установка параметров материала включает в себя привязку текстуры и установку 5 uniform-переменных (diffuse, specular, ambient, shininess и sample), этот процесс будет отнимать достаточно большое количество времени. Вместе с тем, этого можно избежать, поскольку после вывода первого объекта, все эти переменные не менялись и уже имеют нужное значение. Повторная установка параметров материала происходит просто потому, что алгоритм не отслеживает данную ситуацию и каждый раз заново устанавливает все параметры материала.

Вторым недостатком является то, что при выводе третьего объекта произойдет переход к синему материалу, после чего снова возврат к красному материалу, когда потребуется вывести четвертый объект. Красный материал уже был когда-то установлен, но теперь его придется устанавливать заново. Тоже самое касается и повторного возврата к синему материалу при выводе последнего объекта.

Для решения проблемы повторной установки параметров материала, перед непосредственным выводом графических объектов на экран, их необходимо отсортировать по используемым материалам так, чтобы все объекты, использующие один и тот же материал, находились в списке на отрисовку один за другим:

При выводе каждого объекта необходимо отслеживать, какой материал был установлен последним в используемом шейдере и, если этот материал совпадает с тем, который требуется для вывода текущего объекта, не устанавливать заново его параметры. Таким образом, параметры каждого материала буду устанавливаться только один раз при отрисовке каждого кадра изображения.

Данную схему вывода объектов можно проиллюстрировать рисунком, приведенным ниже. Следует, однако, иметь в виду, что помимо параметров материала каждый объект обладает и другими параметрами, являющимися уникальными для этого объекта, например, своей собственной матрицей наблюдения модели. Поэтому часть uniform-переменных все-таки придется устанавливать для каждого объекта:

Для реализации приведенного выше алгоритма оптимизации, который позволяет уменьшить количество обращений к видеокарте, необходимо отсортировать список графических объектов по материалам, текстурам и мешам. Для этого требуется иметь возможность выполнения над объектами этих классов двух основных действий:

1. Прежде всего, требуется определить равенство двух объектов определенных классов. Например, требуется определить, что два графических объекта используют одинаковый материал. Это необходимо для того, чтобы установить, совпадает ли материал для текущего объекта с материалом, который был использован для вывода предыдущего объекта. Если материалы не совпадают, то требуется установить все параметры нового материала, в противном случае эту операцию выполнять не следует, что позволит минимизировать количество обращений к видеокарте.
2. Также необходим способ сравнения двух объектов определенных классов между собой для того, чтобы разместить их в определенном порядке при сортировке. То есть, при сортировке необходимо определить, какой материал должен располагаться раньше, в случае если они не равны. Как правило для этого используют операцию «меньше» и можно говорить о том, что первый материал меньше второго материала, хотя для материалов такое понятие не является очевидным и интуитивно понятным.

За счет того, что работа с материалами, текстурами и мешами выполняется посредством менеджера ресурсов, каждый материал, текстура и меш имеют свой собственный целочисленный идентификатор. То есть можно говорить, например, о том, что графический объект использует материал с идентификатором 1, текстуру с идентификатором 2 и меш с идентификатором 8. В результате сортировка объектов осуществляется исключительно на основании целочисленных значений, без необходимости сравнивать внутреннюю структуру объектов, представляющих материал, текстуру или меш. Сама сортировка является иерархической, то есть:

1. Вначале располагаются графические объекты с меньшем идентификатором материала.
2. Если идентификаторы материала совпадают, то в расчет принимаются идентификаторы текстуры.
3. Если и идентификаторы текстуры совпадают, то в расчет принимаются идентификаторы меша.

Непосредственно для выполнения сортировки рекомендуется использовать функцию sort из стандартной библиотеки шаблонов stl, а не изобретать собственный велосипед. Подробнее с использованием данной функции можно ознакомится, например, по следующим ссылкам:

https://www.youtube.com/watch?v=A3Y0KihfjLY
https://ru.cppreference.com/w/cpp/algorithm/sort

В таблице снизу приведен эффект сортировки объектов по материалам для сцены, которая используется в демонстрационном примере. Сцена содержит 25680 графических объектов, всего один материал и 61 уникальную текстуру. После применения механизмов LoD и Frustum Culling на отрисовку передается 1405 объектов. В результате, просто за счет уменьшения количества обращений к видеокарте, удается добиться определенного прироста производительности.

Используемый алгоритм	**Количество смен материалов/текстур**	**FPS**
Без оптимизации. Материал и текстура устанавливаются для каждого графического объекта.	1405/1405	54
С проверкой идентичности материала и текстуры, но без предварительной сортировки.	1/523	58
С проверкой идентичности материала и текстура, а также с предварительной сортировкой графических объектов по материалам, текстурам и мешам.	1/50	60

Использование geometry instancing'а для уменьшения количества draw call'ов.

Еще одним важным параметром, влияющим на производительность графических приложений, является количество draw call'ов. Draw call – это вызов одной из функций OpenGL, которые непосредственно генерируют полигоны для отрисовки. К таким функциям относятся, например, функции glDrawArrays или glDrawElements. Именно с вызова этих функций начинается основная работа OpenGL: вершины извлекаются из буферов VBO в соответствии с VAO-объектом, передаются конвейеру рендеринга, обрабатываются вершинным шейдером, группируются в полигоны, которые затем разбиваются на фрагменты, для которых вызывается фрагментный шейдер и выполняются пофрагментные операции. Можно сказать, что вызов этих функций запускает конвейер рендеринга.

Функции типа glDrawArrays или glDrawElements приводят к выводу одного меша на экран. Меш – это совокупность полигонов. Чем меньше полигонов в одном меше, тем быстрее он будет выведен на экран. Однако, с точки зрения производительности важно не только количество обрабатываемых вершин, но и количество вызовов этих функций. Так, например, гораздо выгоднее вывести один объект, состоящий из миллиона вершин, чем тысячу объектов, каждый из которых состоит из тысячи полигонов.

Для уменьшения количества draw call'ов существуют несколько способов. Одним из них является группировка нескольких мешей в один. Например, вместо вывода пяти камней, каждый из которых представлен собственным мешем, можно объединить их в один меш, выводя на экран одну «группу камней». Данный подход хорошо работает для случаев, если объекты не меняют взаимного расположения. Перемещение или удаление одного из камней приведет к необходимости пересчитать общий меш.

Второй способ заключается в том, чтобы вывести несколько объектов (несколько мешей) за один draw call. В частности, для любой сложной сцены всегда можно найти несколько одинаковых моделей, которые выгоднее выводить вместе за один раз, нежели выводить каждый объект индивидуально. Данная техника получила название Geometry Instancing (геометрический инстансинг) или просто instancing. Суть метода заключается в том, чтобы вывести на экран несколько копий одного меша за один draw call.

Для поддержки механизма instancing'а в OpenGL были добавлены несколько функций, используемых для вывода мешей. В частности, это функции glDrawArraysInstanced и glDrawElementsInstanced, которые являются аналогами функций glDrawArrays и glDrawElements. Единственным отличием этих функций является наличие еще одного параметра (instancecount), который задает, сколько раз необходимо вывести меш. Подробнее про эти функции можно прочитать, например, по следующим ссылкам:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glDrawArraysInstanced.xhtml
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glDrawElementsInstanced.xhtml

В рамках данного проекта за вывод мешей отвечает класс Mesh, поэтому для реализации техники instancing'а необходимо добавить к нему метод, позволяющий выводить не одну копию меша, как это было сделано ранее, а несколько копий за один раз. Все остальные поля и методы остаются неизменными, поэтому изменения будут незначительными:

```
// КЛАСС ДЛЯ РАБОТЫ С МЕШЕМ
class Mesh
{
public:
        ...
        // вывод меша
        void drawOne();
        // вывод инстанцированного меша (вывести count мешей)
        void drawMany(int count);

private:
        ...

};
```

Используя данный класс, можно вывести один меш сразу несколько раз, то есть каждая вершина этого меша будет передана видеокарте и обработана вершинным шейдером указанное количество раз. При этом возникает резонный вопрос, если каждая из этих копий будет обработан одним и тем же шейдером с

использованием одних и тех же uniform-переменных, то в чем будет разница между всеми этими копиями? Не будут ли они выведены в одном и том же месте так, что разница будет абсолютно незаметна?

В целом, если не предпринимать никаких дополнительных действий, так оно и будет. Для решения этой проблемы в вершинном шейдере доступна одна предопределенная переменная, объявленная следующим образом: *in int gl_InstanceID;*

Данная переменная доступна только в вершинном шейдере и содержит номер экземпляра выводимого меша (начиная с нуля) для команд типа glDrawArraysInstanced и glDrawElementsInstanced. То есть, если данные функции были вызваны с последним параметром 5, то есть с указанием того, что необходимо отобразить пять экземпляров соответствующего меша, то каждая вершина меша будет обработана вершинным шейдером пять раз, но в каждом конкретном случае переменная gl_InstanceID будет принимать значения от нуля до четырех. Данная переменная используется для того, чтобы в вершинном шейдере можно было различить разные копии выводимого меша и обработать их по-разному, например, с использованием разных uniform-переменных.

Таким образом, необходимо модифицировать вершинный шейдер, чтобы он использовал переменную gl_InstanceID и выводил разные копии меша в разных местах. Для этого нужно, чтобы в шейдер передавалась не одна матрица наблюдения модели, а несколько, по одной для каждой копии меша. Сделать это удобнее всего, определив массив матриц наблюдения модели. Полный текст вершинного шейдера, с расчетом, что одновременно будут выводится не более пяти копий меша, приведен ниже:

```
1  #version 330 core
2
3  uniform mat4 projectionMatrix;
4  uniform mat4 modelViewMatrix[5];
5
6  layout(location = 0) in vec3 vPosition;
7  layout(location = 1) in vec3 vNormal;
8  layout(location = 2) in vec2 vTexCoord;
9
10 out vec3 position;
11 out vec3 normal;
12 out vec2 texCoord;
13
14 void main ()
15 {
16     position = vec3(modelViewMatrix[gl_InstanceID] * vec4 (vPosition, 1));
17     normal = vec3(modelViewMatrix[gl_InstanceID] * vec4 (vNormal, 0));
18     texCoord = vec2(vTexCoord.s, 1.0 - vTexCoord.t);
19
20     gl_Position = projectionMatrix * modelViewMatrix[gl_InstanceID] * vec4 (vPosition, 1);
21 }
```

Следует отметить, что фрагментный шейдер при этом не меняется, он по-прежнему накладывает текстуру на фрагмент и вычисляет его цвет по формуле Фонга, основываясь на параметрах, полученных из вершинного шейдера, совершенно игнорируя тот факт, что были выведены несколько копий одного и того же меша. Более того, во фрагментном шейдере переменная gl_InstanceID вообще не доступна.

В рамка данного проекта за взаимодействие с шейдером, в частности, за установку uniform-переменных, отвечает класс Shader, к которому необходимо добавить метод для установки не одной, а сразу нескольких матриц. Данный класс, в итоге, может принимать следующую форму:

```
// КЛАСС ДЛЯ РАБОТЫ С ШЕЙДЕРОМ
class Shader
{
public:
        ...
        // установка uniform-переменных
        void setUniform(std::string name, std::vector<glm::mat4> &value);

private:
        ...

};
```

Непосредственно для установки значение массива матриц в вершинном шейдере, также как и для установки одной матрицы, используется функция glUniformMatrix4fv, одним из параметров которой является количество устанавливаемых матриц. Перед установкой матрицы, возможно, потребуется распаковать данные из vector<mat4> в обычный массив GLfloat[], поскольку функции OpenGL работают только с обычными массивами и ничего не знают о контейнерах stl.

Необходимо отметить, что хотя шейдер и рассчитан на одновременный вывод максимум пяти объектов (поскольку размер массива матриц в шейдере равен пяти), количество объектов может быть меньше. Например, можно установить значение только первых трех матриц и вызвать функцию для отрисовки трех одинаковых мешей. В этом случае вышеприведенный шейдер будет использовать только первые три матрицы и будут выведены только три объекта. Также следует упомянуть, что существует ограничение на максимальное количество uniform-переменных, поэтому передать в шейдер очень большое количество матриц не получится. Максимальное количество uniform-переменных можно получить, используя функцию glGetIntegerv с необходимым параметром:

https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glGet.xhtml

Для эффективного использования техники инстансинга, то есть для вывода одновременно нескольких копий одной модели, необходимо сгруппировать выводимые объекты по используемым материалам, текстурам и мешам. Для примера, приведем схематичное изображение списка объектов, которые необходимо вывести в текущем кадре:

Сортировка должна осуществляться как по материалам/текстурам (чтобы минимизировать количество обращений к видеокарте, а также для того, чтобы установить общий материал/текстуру для всех копий выводимых мешей), так и по мешам, поскольку иногда разные меши используют один и тот же материал/текстуру. При этом более приоритетной является сортировка по материалам. В результате сортировки выводимых объектов для их вывода применяется следующая последовательность действий:

Используя технику инстансинга, можно уменьшить количество draw call'ов, выводя несколько одинаковых мешей за один вызов. Следует отметить, что это стало возможно только благодаря предварительной сортировке объектов по материалам, текстурам и мешам:

Таким образом, используя сортировку по материалам, текстурам и мешам, а также механизм инстансинга, можно реализовать более эффективный алгоритм вывода всех графических объектов, переданных классу RenderManager для отрисовки. Детали алгоритма, такие как необходимые дополнительные переменные, вспомогательные методы и прочие нюансы реализации, необходимо проработать самостоятельно.

Общая схема может быть представлена следующей последовательностью шагов:

1. Выполнить предварительную сортировку списка графических объектов по материалам, текстурам и мешам. Необходимо учитывать, что приоритетной является сортировка по материалам, затем по текстурам и только в случае равенства материалов и текстур в расчет принимается меш. Это позволит минимизировать обращение к видеокарте и одновременно дает возможность использования механизма geometry instancing'a.

2. Выбрать используемый шейдер и установить параметры, общие для всех объектов сцены. К таким параметрам относится матрица проекции и параметры источника света.

3. Вывести все объекты, используя следующую последовательность действий:

   a. Если материал текущего объекта отличается от материала, который был ранее установлен, то необходимо установить параметры нового материала, в противном случае (если материалы совпадают) делать этого не следует.

   b. Если текстура текущего объекта отличается от текстуры, которая была установлена ранее, то необходимо установить новую текстуру, в противном случае (если текстуры совпадают) делать этого не следует.

   c. Вычислить и запомнить матрицу наблюдения модели для текущего графического объекта, то есть происходит накопление матриц наблюдения модели в некотором локальном списке (vector<mat4>). Данный список будет использоваться в дальнейшем для передачи в GLSL-шейдер массива матриц при выводе сразу нескольких одинаковых мешей.

   d. Если уже накопили достаточное количество одинаковых моделей (например, пять моделей) или следующая модель отличается мешем, материалом или текстурой, то необходимо вывести накопленные модели. Для этого устанавливается нужное количество матриц наблюдения модели и вызывается метод отрисовки меша, который генерирует вывод сразу нескольких его копий. После это накопление матриц начинается заново.

В результате применения вышеописанного алгоритма оптимизации удалось дополнительно увеличить количество FPS с 60 до 63, а также уменьшить количество drawcall’ов с 1405 до 325:

Задание к лабораторной работе.

Лабораторная работа №7 строится на основе предыдущей работы. К программе предъявляются следующие требования:

1. Необходимо внести изменения в класс GraphicObject в соответствии с описанием, приведенном в данной лабораторной работе. Требуется, прежде всего, добавить к классу поля для представления типа графического объекта и его габаритных размеров.
2. Требуется реализовать класс Scene для работы со сценой. Класс должен иметь структуру, приведенную в соответствующем разделе. Необходимо, прежде всего, обратить внимание на метод createGraphicObject, позволяющий создавать один графический объект в соответствии с описанием в json-файле, а также на метод loadFromJson, загружающий всю сцену (все её графические объекты) из json-файла.
3. В классе Scene необходимо реализовать метод lodTest, который осуществляет проверку на то, нужно ли выводить объект на экран в зависимости от его типа и дистанции до наблюдателя.
4. Также в классе Scene требуется реализовать метод frustumCullingTest, который определяет, попадает ли объект в усеченную пирамиду видимости.
5. В классе Scene необходимо реализовать подсчет общего количества объектов сцены, а также количество объектов сцены, переданное на отрисовку. Данную информацию необходимо выводить в заголовке окна так, как это сделано в демонстрационном примере.
6. В классе RenderManager необходимо организовать сортировку графических объектов по материалам, а также проверку на необходимость смены материала и текстуры при выводе очередного графического объекта, что позволит уменьшить количество обращений к видеокарте.
7. В классе RenderManager необходимо также реализовать geometry instancing, что позволит выводить несколько одинаковых моделей за один раз.
8. Класс RenderManager должен подсчитывать количество смен материалов и текстур, а также количество drawcall’ов, потраченных на построение одного кадра. Указанную информацию необходимо выводить в заголовке окна так, как это сделано в демонстрационном примере.
9. Ресурсы для демонстрации программы (меши, текстур, материал, шейдеры и сцены) необходимо взять из демонстрационного примера. Для демонстрации эффекта алгоритмов оптимизации следует использовать самую большую сцену, описанную в файле «big_scene.json».

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

Архитектура графических систем (6 семестр)

Выполнил: Иванов Иван Иванович

Группа: 00-00

Проверил: Галибин Сергей Владимирович

Задание:

В лабораторной работе №7 необходимо организовать вывод на экран сложной сцены, состоящей из нескольких тысяч графических объектов, а так обеспечить их эффективный вывод, задействую несколько алгоритмов оптимизации, среди которых: level of details, frustum culling и geometry instancing. Для работы со сценой требуется создать отдельный класс Scene.

Защита лабораторной работы:

Для защиты лабораторной работы необходимо знать:

1. **[КГ]** В чем заключается смысл применения уровней детализации.
2. **[КГ]** Что такое frustum culling.
3. **[КГ]** В чем заключается смысл сортировки графических объектов по материалам, текстурам и мешам.
4. **[КГ]** Как реализуется geometry instancing и в чем его смысл.

Состав проекта:

Название модуля и его назначение	**Количество строк**	
	***.h**	***.cpp**
main - основная программа		
Shader -		
Camera -		
GraphicObject -		
Mesh -		
ResourceManager -		
Texture -		
RenderManager –		
Material -		
Light -		
Scene -		